1. Project Overview

> *"I don't solve DSA problems. I design systems that naturally require DSA."*

DSA Master Engine is a long-term, production-quality C++ project designed to teach Data Structures and Algorithms through the lens of real system design — not through isolated LeetCode-style exercises. Every data structure, every algorithm, and every core system in this project exists because a real simulation demands it.

DSA Master Engine

Complete Project Documentation

Advanced C++ System Design | FAANG-Level DSA Project

> *"Before writing a single line of code, a senior engineer asks: What is the purpose? What invariants must hold? Who owns this memory? What is the complexity? What is the tradeoff?"*

- A collection of isolated algorithm implementations
- An interview-prep cheat sheet
- A college assignment with print statements
- A clone of std::vector or std::map

- A layered software system built like a production FAANG codebase
- A learning vehicle that forces deep understanding of memory, ownership, and tradeoffs
- A portfolio piece discussable at Google / Meta / Amazon / Apple / Netflix / Microsoft interviews
- A project where every DSA concept is motivated by a real engineering need

Project At a Glance

Language:	C++17
Architecture:	Layered (Config -> Core -> DS -> Algo -> Simulations)
Data Structs:	20+ implementations
Algorithms:	30+ algorithms across 8 categories
Simulations:	Search Engine, Job Recommender, Traffic Router, Analytics

2. System Architecture

The project follows a strict layered architecture. Each layer depends only on the layer below it. Simulations never reach into core internals directly — they use algorithms and data structures. Algorithms never know about simulations. This separation makes every component independently testable and replaceable.

Layer	Directory	Purpose	Depends On
Simulations	simulations/	Real-world problem domains	Algorithms, Data Structures, Core
Algorithms	algorithms/	Pure algorithmic logic	Data Structures
Data Structs	data_structures/	Storage and organization	Standard Library only
Core Systems	core/	Infrastructure: cache, logger...	Config, Standard Library
Config	config/	Constants and system settings	Standard Library only

2.1 Dependency Flow

```
main.cpp
   |
```

```
   v
Simulations   (search_engine, job_recommender, traffic_router, analytics)
   |
   v
Algorithms    (sorting, searching, graph, DP, backtracking, string...)
   |
   v
Data Structs  (array, list, stack, queue, tree, graph, trie, hashmap...)
   |
   v
Core          (logger, cache, scheduler, rate limiter, load balancer)
   |
   v
Config        (constants.h, system_config.h)
```

2.2 Engineering Principles

- Invariant-first design: every class defines what must always be true before writing a single method
- Ownership clarity: every heap allocation has exactly one owner responsible for freeing it
- Complexity awareness: every operation's time and space complexity is known and justified
- No premature abstraction: functionality is added only when a real simulation demands it
- Const-correctness: read-only methods are marked const throughout
- Template discipline: generic containers use templates; infrastructure uses concrete types

3. Complete File Structure

```
DSA-Master-Engine/
|
+-- CMakeLists.txt            # Build system (C++17, MSVC + GCC compatible)
+-- main.cpp                  # Entry point, demos all layers
+-- generate_doc.py           # This documentation generator
|
+-- config/
|   +-- constants.h           # System-wide compile-time constants
|   +-- system_config.h       # SystemConfig struct + singleton accessor
|
+-- core/
|   +-- logger/
|   |   +-- logger.h/.cpp     # Thread-safe singleton logger, LOG_* macros
|   +-- memory_manager/
|   |   +-- lru_cache.h/.cpp  # LRU cache (doubly-linked list + hashmap)
|   |   +-- lfu_cache.h/.cpp  # LFU cache (frequency map + per-freq lists)
|   +-- rate_limiter/
|   |   +-- token_bucket.h/.cpp  # Token bucket rate limiter
|   +-- load_balancer/
|   |   +-- server.h          # Server struct
|   |   +-- round_robin.h/.cpp
|   |   +-- least_connection.h/.cpp
|   +-- schedular/
|       +-- task.h            # Task struct with priority and work function
|       +-- task_schedular.h/.cpp  # Max-heap priority scheduler
|
+-- data_structures/
|   +-- array/dynamic_array.h
|   +-- linked_list/singly_ll.h, doubly_ll.h, circular_ll.h
|   +-- stack/stack_array.h
|   +-- queue/simple_queue.h, circular_queue.h, deque.h
|   +-- hashmap/hashmap_chaining.h, hashmap_open_addressing.h
|   +-- tree/binary_tree.h, bst.h, avl_tree.h, red_black_tree.h
|   +-- tree/heap.h, segment_tree.h, fenwick_tree.h
|   +-- trie/trie.h
|   +-- graph/graph.h, disjoint_set.h
|
+-- algorithms/
|   +-- sorting/      bubble, insertion, selection, merge, quick,
|   |                 heap, counting, radix
|   +-- searching/    linear, binary, exponential
|   +-- greedy/       activity_selection, huffman_coding
|   +-- divide_conquer/ merge_sort_dc, max_subarray_sum
|   +-- bactracking/  n_queens, sudoku_solver
|   +-- dynamic_programming/ knapsack, lcs, lis, matrix_chain
|   +-- string_algorithms/   kmp, rabin_karp, z_algorithm, suffix_array
|   +-- graph_algorithms/    bfs, dfs, dijkstra, bellman_ford,
|                            floyd_warshall, kruskal, prim, topo_sort
|
+-- simulations/
    +-- search_engine/   search_engine.h/.cpp
    +-- job_recommender/ job_matcher.h/.cpp, recommendation_engine.h/.cpp
    +-- traffic_router/  traffic_router.h/.cpp
    +-- analytics/       sliding_window_metrics.h/.cpp
```

4. Data Structures

All data structures are implemented as header-only C++ templates (except FenwickTree and Trie which are non-template). Each is designed with explicit invariants that every method must maintain.

4.1 DynamicArray

A heap-allocated, contiguous-memory, growable sequence container — the foundation of the project and the first data structure taught. It is NOT a std::vector clone; it exists to teach memory management, ownership, amortized analysis, and object lifetime.

Internal State

```
T*     data_;      // owning raw pointer to heap allocation
size_t size_;      // number of live elements
size_t capacity_;  // number of allocated slots
```

Invariants

- size_ <= capacity_ (always)
- capacity_ == 0 => data_ == nullptr
- capacity_ > 0 => data_ != nullptr

Operation	Complexity	Reason
push_back	Amortized O(1)	Geometric growth (x2): total copies over N pushes = 2N
pop_back	O(1)	Just decrements size_. Allocation untouched.
operator[]	O(1)	Direct pointer arithmetic: base + index * sizeof(T)
at()	O(1)	Bounds check + pointer arithmetic. Throws out_of_range.
reserve()	O(n)	Allocate new block, copy n elements, free old block.
clear()	O(1)	Sets size_ = 0. Keeps allocation alive for reuse.

Key Design Tradeoff

- operator[] — no bounds check. Used in tight loops (e.g. ranking engine). Performance over safety.
- at() — bounds check, throws. Used at system boundaries (e.g. user-provided indices).
- Growth by x2 not x1: at 1M elements, x1 costs O(N^2) copies total; x2 costs O(N) total.

4.2 SinglyLinkedList

A chain of heap-allocated nodes connected through pointers. Teaches the opposite mental model from DynamicArray: non-contiguous memory, O(1) insert anywhere (with pointer), O(n) index access. Used as the task queue in the Task Scheduler.

Node Structure

```
struct Node {
    T     data;
    Node* next;    // address of next node, nullptr if last
};
```

Invariants

- head_ == nullptr => tail_ == nullptr => size_ == 0
- size_ == 1 => head_ == tail_
- tail_->next == nullptr (always)

Operation	Complexity	Notes
insert_head	O(1)	Rewire one pointer + update head_
insert_tail	O(1)	tail_ gives direct access
delete_head	O(1)	Rewire head_, handle empty result
delete_tail	O(n)	Must walk to find second-to-last — singly list limitation
delete_by_value	O(n)	Walk with prev/current two-pointer pattern
search	O(n)	No index; must follow pointer chain

The O(n) delete_tail is the fundamental limitation of a singly linked list — you cannot go backwards without a prev pointer. This directly motivates DoublyLinkedList.

4.3 DoublyLinkedList

Extends singly linked list with a prev pointer per node. This single addition makes delete_tail O(1) and enables the move_to_front operation critical for LRU cache. Tradeoff: 8 extra bytes per node on 64-bit systems.

- Used internally by LRU Cache for O(1) eviction of least-recently-used entry
- move_to_front(node*) — rewires prev/next in O(1), used on every cache hit
- delete_tail() — O(1) via tail_->prev, the key upgrade over singly linked list

4.4 Stack

Built on top of DynamicArray. Demonstrates the abstraction layer: the user sees push/pop/top with no awareness of the underlying heap management. This is the stack used for DFS traversal in graph algorithms.

Method	Delegates To	Complexity
push(val)	data_.push_back(val)	Amortized O(1)
pop()	data_.pop_back()	O(1)
top()	data_[size-1]	O(1)

4.5 Queue Variants

- SimpleQueue — std::deque backed, unbounded, used in BFS traversal
- CircularQueue — fixed-capacity ring buffer, O(1) enqueue/dequeue, wrap-around index arithmetic
- Deque — growable ring buffer with push/pop at both ends, used in sliding window analytics

4.6 HashMap (Two Implementations)

Both implementations map keys to values using hash functions. They differ in collision resolution strategy — a fundamental tradeoff in systems design.

	HashMapChaining	HashMapOpenAddressing
Collision handling	Linked list per bucket	Linear probing + tombstones
Memory layout	Non-contiguous (pointers)	Contiguous array
Cache performance	Poor (pointer chasing)	Good (sequential access)
Load factor	Can exceed 1.0	Must stay below 0.5
Deletion	Remove from list O(1)	Mark as DELETED tombstone
Used when	Simple, flexible	Performance-critical paths

4.7 Tree Data Structures

BST (Binary Search Tree)

Maintains the BST invariant: left subtree < node < right subtree. Average case O(log n) for insert/search/delete. Worst case O(n) on sorted input — motivates AVL tree.

AVL Tree

Self-balancing BST. Maintains balance factor = height(left) - height(right) in {-1, 0, 1}. Performs single or double rotations on insert to restore balance. Guarantees O(log n) worst case for all operations.

Rotation Type	When Triggered	Result
Right rotate	Left-heavy (balance > 1, left child balanced/left-heavy)	Reduces left subtree height
Left rotate	Right-heavy (balance < -1, right child balanced/right-heavy)	Reduces right subtree height
Left-Right rotate	Left-heavy, left child is right-heavy	Double rotation to fix
Right-Left rotate	Right-heavy, right child is left-heavy	Double rotation to fix

Red-Black Tree

A self-balancing BST with 5 invariants: nodes are red or black, root is black, red nodes have black children, all paths from a node to leaves have equal black-height, leaves (NIL sentinels) are black. Less strictly balanced than AVL — O(log n) guaranteed but with fewer rotations on insert.

Heap (MaxHeap / MinHeap)

A complete binary tree stored in an array. Parent is always >= children (max-heap). Enables O(1) access to maximum element and O(log n) insert/delete. Used as the priority queue backbone in TaskScheduler and Dijkstra's algorithm.

```
// Parent-child index relationships in array representation:
parent(i) = (i - 1) / 2
left(i)   = 2 * i + 1
right(i)  = 2 * i + 2
```

Segment Tree

Enables O(log n) range queries (sum, min, max, GCD) and O(log n) point updates on an array. Parameterized by a combine function and identity element — the same tree structure handles any associative operation. Used in analytics for range aggregation over time windows.

Fenwick Tree (Binary Indexed Tree)

A space-efficient alternative to segment tree for prefix sum queries. O(log n) update, O(log n) prefix sum using bit manipulation. Used in the analytics layer for cumulative metric tracking.

Trie

A tree where each node represents a character. Enables O(m) insert/search (m = word length), prefix queries, and autocomplete in O(m + results). Used in the Search Engine for fast prefix-based document lookup.

5. Algorithms

All algorithms are implemented as standalone functions in .cpp files with corresponding .h declaration files. No algorithm file contains a main() function. Each algorithm is motivated by a real need within the simulations.

5.1 Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable	When to Use
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Teaching only — never production
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Nearly-sorted data, small n
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No	When writes are expensive
Merge Sort	O(n logn)	O(n logn)	O(n logn)	O(n)	Yes	Stable sort, linked lists
Quick Sort	O(n logn)	O(n logn)	O(n^2)	O(logn)	No	General purpose, best cache behaviour
Heap Sort	O(n logn)	O(n logn)	O(n logn)	O(1)	No	When guaranteed O(n logn) + O(1) space
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	Yes	Integer keys with small range k
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)	Yes	Large integers, fixed-length strings

Project Connection: Search Engine Ranking

When the Search Engine returns results, they are sorted by relevance score using std::sort (introsort — hybrid of quicksort + heapsort + insertion sort). Understanding when to use each sorting algorithm is what allows you to choose the right tool when implementing a custom ranking pipeline.

5.2 Searching Algorithms

Algorithm	Complexity	Requirement	Project Use
Linear Search	O(n)	Unsorted array	Fallback search in small datasets
Binary Search	O(log n)	Sorted array	Searching sorted job listings by salary
Exponential Search	O(log n)	Sorted array	Unknown-size sorted stream searching

5.3 Graph Algorithms

Algorithm	Complexity	Purpose in Project
BFS	O(V + E)	Level-order traversal, shortest hops in unweighted graph
DFS	O(V + E)	Cycle detection, connectivity, topological ordering
Dijkstra	O((V+E) log V)	Min-latency routing in TrafficRouter simulation
Bellman-Ford	O(V * E)	Negative-weight edge detection in routing
Floyd-Warshall	O(V^3)	All-pairs shortest paths for small routing tables
Kruskal (MST)	O(E log E)	Minimum spanning tree using DisjointSet union-find
Prim (MST)	O(E log V)	Alternative MST — better for dense graphs
Topological Sort	O(V + E)	Task dependency ordering in Scheduler

Project Connection: Traffic Router

The TrafficRouter simulation uses Dijkstra's algorithm to find the minimum-latency path between network nodes. The Graph data structure (adjacency list) backs the router's network topology. Kruskal's algorithm with DisjointSet union-find is used to compute the minimum spanning tree — the backbone of network design.

5.4 Dynamic Programming

Problem	Technique	Complexity	Project Application
0/1 Knapsack	Bottom-up 2D DP	O(n*W)	Resource allocation in Scheduler
LCS	Bottom-up 2D DP	O(m*n)	Document similarity in Search Engine
LIS	Patience sorting	O(n log n)	Version sequence detection
Matrix Chain	Interval DP	O(n^3)	Query plan optimization

5.5 Backtracking

Problem	Approach	Complexity
N-Queens	Place queen row by row; backtrack on conflict	O(n!)
Sudoku	Fill empty cells 1-9; backtrack on invalid	O(9^(n*n))

5.6 String Algorithms

Algorithm	Preprocessing	Search	Key Idea
KMP	O(m)	O(n)	Failure function avoids re-examining matched characters
Rabin-Karp	O(m)	O(n)	Rolling hash; compare hash before string comparison

Algorithm	Preprocessing	Search	Key Idea
Z-Algorithm	O(n)	O(n)	Z-array: longest prefix match starting at each position
Suffix Array	O(n log n)	O(m logn)	Sorted array of all suffixes enables binary search

Project Connection: Search Engine

KMP is used in the Search Engine for exact keyword matching within documents. Rabin-Karp provides rolling-hash multi-pattern search. The Trie structure handles prefix queries and autocomplete. Together these form a complete text search pipeline.

6. Core Systems

Core systems are infrastructure components that every serious production system needs before any business logic. The mentor principle: every real system begins with configuration, visibility (logging), and diagnostics.

6.1 Logger

A thread-safe singleton logger with four log levels: DEBUG, INFO, WARN, ERROR. Uses std::mutex to prevent interleaved output from concurrent systems. Provides convenience macros LOG_INFO(), LOG_WARN(), LOG_ERROR(), LOG_DEBUG().

```
// Usage:
LOG_INFO("Scheduler: submitted task [" + task.name + "]");
LOG_ERROR("Rate limiter: request denied — bucket empty");
```

- Singleton pattern — one logger instance for the entire system
- Level filtering — DEBUG messages suppressed in production
- Optional file output — appends to log file if enabled
- Thread-safe — std::lock_guard on every write operation

6.2 LRU Cache

Caches the most recently used items and evicts the least recently used when capacity is exceeded. Used in the Search Engine to cache query results so repeated queries don't re-scan the inverted index.

Data Structure Combination

```
std::list>                          // doubly linked list
    front = most recent  |  back = LRU (evict target)

std::unordered_map         // O(1) lookup
    key -> pointer into the list
```

Operation	Complexity	How
get(key)	O(1)	hashmap lookup + splice node to front
put(key,val)	O(1)	hashmap insert + list push_front; evict back if full
evict()	O(1)	erase list.back(), remove from hashmap

6.3 LFU Cache

Evicts the least frequently used item. When multiple items share minimum frequency, evicts the least recently used among them. Used for hot content caching where access frequency determines retention priority.

- key_map: key -> list iterator (O(1) access to entry)
- freq_map: frequency -> doubly linked list of entries at that frequency
- min_freq: tracks the current minimum frequency for O(1) eviction
- On access: move entry to freq+1 list; update min_freq if freq list becomes empty

6.4 Token Bucket Rate Limiter

Limits API request rate. Tokens accumulate at a fixed rate (e.g. 10/sec) up to a maximum capacity. Each request consumes one token. Burst requests are allowed up to bucket capacity; sustained rate is capped.

```
TokenBucket limiter(10.0, 50);   // 10 tokens/sec, capacity 50

if (limiter.consume(1)) {
    handle_request();            // allowed
} else {
    return 429_TOO_MANY_REQUESTS; // denied
}
```

6.5 Load Balancer

Strategy	Algorithm	Best For	Limitation
Round Robin	Cycle through servers in order	Equal-cost homogeneous servers	Ignores actual load
Least Connection	Route to server with fewest active connections	Variable request cost	Slightly more overhead

6.6 Task Scheduler

A priority-based task executor. Tasks are submitted with an integer priority. The scheduler always runs the highest-priority pending task next (max-heap). Supports cancellation via a cancellation map — cancelled tasks are skipped at dequeue time.

```
scheduler.submit(Task(1, "Compress logs",    priority=2, work));
scheduler.submit(Task(2, "Index documents",  priority=5, work));
scheduler.submit(Task(3, "ALERT: disk full", priority=9, work));
scheduler.cancel(1);    // cancel task 1
scheduler.run_all();    // runs in order: 9, 5  (task 1 skipped)
```

7. Simulations

Simulations are the top layer of the project — the real-world problem domains that justify every data structure and algorithm below them. A simulation does not implement algorithms directly; it orchestrates the layers beneath it.

7.1 Search Engine

A document indexing and retrieval system with TF-inspired relevance scoring.

Component	Data Structure / Algorithm	Purpose
Inverted Index	HashMapChaining>	Map term -> list of document IDs
Document Store	unordered_map	O(1) document retrieval by ID
Tokenizer	string parsing	Normalize text to lowercase terms
Ranking	std::sort on relevance score	Sort results by TF + title-match bonus
Query Cache	LRUCache>	Cache repeated query results
Autocomplete	Trie	Prefix-based term suggestions

7.2 Job Recommender

Matches job listings to candidates based on skill overlap and salary requirements.

- Candidate provides: skill set, minimum salary requirement
- Each job has: required skills, offered salary, company
- Match score = |candidate_skills ∩ job_skills| / |job_skills| (Jaccard-like coefficient)
- Results filtered by salary >= min_salary, sorted by match score descending
- 8 hardcoded FAANG/top-tech jobs pre-loaded (Google, Meta, Amazon, OpenAI, Apple, Netflix, Microsoft, Cloudflare)

7.3 Traffic Router

Models a network topology as a weighted undirected graph. Finds the minimum-latency path between any two nodes using Dijkstra's algorithm with a priority queue (min-heap). Reconstructs the full path via parent array.

```
TrafficRouter router(6);          // 6 network nodes
router.add_link(0, 2,  3);        // Node0 <-> Node2: 3ms
router.add_link(2, 4,  2);        // Node2 <-> Node4: 2ms
router.add_link(4, 5,  1);        // Node4 <-> Node5: 1ms

auto route = router.find_best_route(0, 5);
// Path: Node0 -> Node2 -> Node4 -> Node5
// Latency: 6ms
```

7.4 Analytics — Sliding Window Metrics

Tracks real-time system metrics (latency, throughput, error rate) over a configurable time window using a deque-backed sliding window.

- Deque maintains events in chronological order — front = oldest, back = newest
- evict(current_time) — pops from front while oldest event is outside the window
- average(), max(), min(), count() — computed over the current window contents
- Used to track search engine query latency and recommendation engine response times

8. Configuration System

8.1 constants.h

Compile-time constants defined in the DME namespace. All magic numbers in the system reference these constants rather than raw literals.

```
namespace DME {
    constexpr size_t DEFAULT_CACHE_CAPACITY  = 100;
    constexpr double DEFAULT_TOKEN_RATE       = 10.0;  // tokens/sec
    constexpr size_t MAX_SERVERS              = 32;
    constexpr size_t SLIDING_WINDOW_SIZE      = 60;    // seconds
    constexpr size_t MAX_SEARCH_RESULTS       = 20;
}
```

8.2 system_config.h

A runtime-configurable SystemConfig struct with a singleton accessor. Subsystems query get_config() to read their operating parameters. This allows the same binary to run in different configurations (e.g. larger cache for production, smaller for test).

9. Build System

9.1 CMakeLists.txt

- CMake 3.16+ required
- C++17 standard enforced (std::optional, structured bindings, if-constexpr)
- MSVC warnings: /W3 /wd4996
- GCC/Clang warnings: -Wall -Wextra
- Release: /O2 (MSVC) or -O3 (GCC/Clang)
- Debug: /Od /Zi (MSVC) or -O0 -g (GCC/Clang)
- All .cpp files discovered via file(GLOB_RECURSE) — no manual listing needed
- Test executable built only if tests/*.cpp files exist (conditional)

9.2 Build Instructions

Visual Studio (Windows — Recommended)

```
# Open 'Developer Command Prompt for VS 2022'
cd D:\DME
cmake -B build
cmake --build build --config Release
build\Release\dsa_engine.exe
```

MinGW / GCC (Windows)

```
cd D:\DME
cmake -B build -G "MinGW Makefiles" -DCMAKE_BUILD_TYPE=Release
cmake --build build
.\build\dsa_engine.exe
```

GCC (Linux / macOS)

```
cd /path/to/DME
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
./build/dsa_engine
```

10. Engineering Philosophy

> *"Do not memorize algorithms. Build systems that require algorithms."*

10.1 Invariant-First Design

Before implementing any data structure, define the invariants — the rules that must be true at all times. Every method must maintain them. Breaking an invariant, even temporarily inside a method, is a bug.

- DynamicArray: size_ <= capacity_ always
- SinglyLinkedList: tail_->next == nullptr always
- AVLTree: |balance_factor| <= 1 at every node always
- Heap: parent >= children (max-heap) at every node always

10.2 Ownership Model

In C++ without a garbage collector, every heap allocation must have exactly one owner responsible for freeing it. The project teaches three levels:

Level	Pattern	Example in Project
Raw pointer	Manual new/delete in class	DynamicArray::data_, all tree nodes
RAII	Constructor allocates, destructor frees	Every class with heap state
Shared ownership	std::shared_ptr	Huffman tree nodes in huffman_coding.cpp

10.3 Amortized Analysis

An operation that is occasionally expensive but cheap on average. DynamicArray::push_back doubles capacity when full. Each reallocation copies N elements, but this happens so rarely that averaged over N pushes the cost is O(1) per operation.

```
N pushes with capacity doubling:
   Copy counts:  1 + 2 + 4 + 8 + ... + N  =  ~2N total copies
   Cost per push: 2N / N  =  O(1) amortized

N pushes with capacity + 1 each time:
   Copy counts:  0 + 1 + 2 + ... + N-1  =  N*(N-1)/2  =  O(N^2) total
   Cost per push: O(N)  --  UNACCEPTABLE at scale
```

10.4 The Senior Engineer Mindset

For every data structure and algorithm, a senior engineer asks:

Question	What It Reveals
What is the purpose?	Is this component justified? Could something simpler work?
What invariants must hold?	What assumptions does every caller rely on?
Who owns this memory?	Which destructor frees this allocation?

Question	What It Reveals
What is the complexity?	Will this scale to 10x, 100x, 1000x the current load?
What is the tradeoff?	What does this design sacrifice to gain its advantages?
What breaks at the boundary?	What happens with empty input, max input, duplicate input?

10.5 Learning Progression

The data structures are built in a deliberate order. Each one teaches a concept that the next one builds upon.

Order	Structure	Core Concept Taught
1	DynamicArray	Contiguous memory, ownership, amortized O(1), Rule of 5
2	SinglyLinkedList	Pointers, non-contiguous memory, structural invariants
3	Stack	Abstraction: hide implementation behind clean interface
4	SimpleQueue	State transitions, FIFO discipline
5	CircularQueue	Index wrap-around, fixed-capacity ring buffer
6	Deque	Bidirectional operations, growable ring buffer
7	HashMap	Hashing, collision resolution, load factor
8	BST	Recursive structure, O(log n) average case
9	AVL Tree	Self-balancing, rotations, O(log n) worst case
10	Heap	Complete tree in array, sift operations, priority
11	Trie	Character-level indexing, prefix operations
12	Graph	Vertex-edge relationships, adjacency representation

11. Quick Reference — Complexity Cheat Sheet

11.1 Data Structures

Structure	Access	Search	Insert	Delete	Space
DynamicArray	O(1)	O(n)	O(1)*	O(n)	O(n)
SinglyLinkedList	O(n)	O(n)	O(1)	O(n)	O(n)
DoublyLinkedList	O(n)	O(n)	O(1)	O(1)	O(n)
Stack	O(1)top	O(n)	O(1)*	O(1)	O(n)
Queue	O(1)frnt	O(n)	O(1)	O(1)	O(n)
HashMap	O(1)*	O(1)*	O(1)*	O(1)*	O(n)
BST	O(logn)	O(logn)	O(logn)	O(logn)	O(n)
AVL Tree	O(logn)	O(logn)	O(logn)	O(logn)	O(n)
Heap	O(1)top	O(n)	O(logn)	O(logn)	O(n)
Trie	O(m)	O(m)	O(m)	O(m)	O(n*m)
Segment Tree	-	O(logn)	O(logn)	-	O(n)
Fenwick Tree	-	O(logn)	O(logn)	-	O(n)

* = amortized or average case

11.2 Sorting Algorithms

Algorithm	Best	Average	Worst	Space
Bubble	O(n)	O(n^2)	O(n^2)	O(1)
Insertion	O(n)	O(n^2)	O(n^2)	O(1)
Selection	O(n^2)	O(n^2)	O(n^2)	O(1)
Merge	O(nlogn)	O(nlogn)	O(nlogn)	O(n)
Quick	O(nlogn)	O(nlogn)	O(n^2)	O(logn)
Heap	O(nlogn)	O(nlogn)	O(nlogn)	O(1)
Counting	O(n+k)	O(n+k)	O(n+k)	O(k)
Radix	O(nk)	O(nk)	O(nk)	O(n+k)

11.3 Graph Algorithms

Algorithm	Time	Space	Notes
BFS	O(V+E)	O(V)	Shortest path (unweighted)

Algorithm	Time	Space	Notes
DFS	O(V+E)	O(V)	Cycle detection, topological sort
Dijkstra	O((V+E)logV)	O(V)	Non-negative weights only
Bellman-Ford	O(VE)	O(V)	Handles negative weights
Floyd-Warshall	O(V^3)	O(V^2)	All-pairs shortest paths
Kruskal MST	O(E logE)	O(V)	Requires DisjointSet
Prim MST	O(E logV)	O(V)	Better for dense graphs
Topo Sort	O(V+E)	O(V)	DAG only

12. Glossary

Amortized O(1): Average cost per operation over a sequence of N operations is O(1), even if some individual operations cost O(N).

Invariant: A condition that must be true at all times. Violation means a bug.

RAII: Resource Acquisition Is Initialization — constructor acquires resource, destructor releases it. Guarantees no leaks.

Contiguous Memory: Elements stored at consecutive memory addresses. Enables O(1) index access via pointer arithmetic.

Cache Locality: CPU caches work on cache lines (~64 bytes). Contiguous data is loaded together, making sequential access fast. Linked lists break locality.

Load Factor: ratio = elements / buckets in a hash table. High load factor increases collisions; triggers rehashing.

Sentinel Node: A dummy node (e.g. NIL in Red-Black Tree) that simplifies boundary conditions. Eliminates nullptr checks.

Sift Up/Down: Heap operations to restore heap property after insert (sift up) or delete (sift down).

Balance Factor: AVL Tree: height(left) - height(right). Must stay in {-1, 0, 1}.

Tombstone: A marker in open-addressing hash tables indicating a deleted slot. Allows search to continue past it without treating it as empty.

Path Compression: DisjointSet optimization: during find(), make every node on the path point directly to the root. Amortizes future finds to ~O(1).

Union by Rank: DisjointSet optimization: always attach smaller tree under larger. Keeps tree height at O(log n).

Inverted Index: Maps each term to the list of documents containing it. Core data structure of every search engine.

Token Bucket: Rate limiting algorithm: tokens accumulate at rate R up to capacity C. Each request consumes one token. Allows bursts up to C.

13. FAANG Interview Discussion Guide

This section maps the project's components to common FAANG system design and DSA interview topics. Use this project as evidence of deep understanding rather than memorized solutions.

13.1 System Design Topics Covered

Topic	Where in This Project
Caching strategies	LRU Cache (recency), LFU Cache (frequency) — both implemented from scratch
Rate limiting	Token Bucket in core/rate_limiter/ — production pattern
Load balancing	Round Robin + Least Connection in core/load_balancer/
Search systems	Inverted index, TF scoring, Trie autocomplete in simulations/search_engine/
Priority scheduling	Max-heap TaskScheduler with cancellation in core/schedular/
Graph routing	Dijkstra-based TrafficRouter in simulations/traffic_router/
Sliding window metrics	Deque-backed SlidingWindowMetrics in simulations/analytics/
Recommendation systems	Skill-match scoring JobRecommender in simulations/job_recommender/

13.2 DSA Depth Questions You Can Answer

- Why does DynamicArray use capacity *= 2 instead of capacity += 1? (Amortized O(1) vs O(N^2))
- Why does delete_tail() cost O(n) in a singly linked list? (No backwards traversal)
- Why does LRU Cache need a doubly linked list + hashmap? (O(1) delete + O(1) lookup)
- What invariant does AVL Tree maintain and how does it enforce it? (Balance factor via rotations)
- Why is Dijkstra's algorithm wrong with negative weights? (Greedy assumption breaks)
- What is the difference between Kruskal and Prim for MST? (Edge-sorted vs vertex-grown)
- Why does the Trie use unordered_map instead of a fixed array? (Sparse alphabet)
- What is path compression in DisjointSet and why does it matter? (Near-O(1) amortized find)
- Why use a Fenwick Tree instead of a Segment Tree for prefix sums? (Half the space, simpler code)
- What is the time complexity of KMP and why? (O(n+m) — failure function prevents backtracking)

13.3 The Core Message

> *"I built a system where data structures and algorithms are not isolated exercises — they are requirements. My search engine needs a Trie. My traffic router needs Dijkstra. My scheduler needs a max-heap. I understand not just how these work, but WHY they exist and what breaks if you choose the wrong one."*